Rapport Projet

-Base de données orientée objet-

Latouche Jules,Duperron Bastien | Info4B | 06/04/2025

UNIVERSITÉ BOURGOGNE EUROPE

Sommaire:

I. Analyse fonctionnelle.
II. Description des structures de données et architecture logicielle.
III. Spécification des classes principales et description des algorithmes principaux.
IV. Jeu de test montrant les fonctionnalités disponibles.
V. Conclusion.

Introduction:

Ce système repose sur une architecture client-serveur. Le serveur, en exécution continue, assure la gestion multi-utilisateurs, l'authentification, la persistance des collections d'objets et la réponse aux requêtes clients. Les clients, exécutés en mode terminal, interagissent avec le serveur à travers des méthodes de fonctionnement, développées spécifiquement pour ce projet. Cette bibliothèque permet d'effectuer l'ensemble des opérations CRUD (Create, Read, Update, Delete) sur des objets distants.

Ce projet a été réalisé dans le cadre du module INFO4B. Il met en œuvre des concepts fondamentaux en programmation orientée objet, en communication réseau, en gestion de la concurrence ainsi qu'en organisation logicielle en couches. Dans ce rapport, on présente de manière structurée les choix techniques effectués, l'architecture du système, les principales classes implémentées, les algorithmes utilisés ainsi qu'un jeu de tests démontrant le bon fonctionnement de l'application.

I. Analyse fonctionnelle

L'application a pour but de :

-Stocker, consulter, ajouter et supprimer des objets dans une base de données persistante orientée objet.

-Gérer plusieurs clients simultanément via un système client-serveur.

-Sécuriser l'accès par un système d'authentification (connexion et enregistrement).

-Offrir une interface utilisateur simple en ligne de commande CRUD pour interagir avec la base.

Authentification :

- L'utilisateur doit d'abord s'enregistrer avec la commande REGISTER;nom;motdepasse ou se connecter avec la commande LOGIN;nom;motdepasse.
- L'accès à la base de données est interdit sans authentification.
- Commande disponibles(après authentification):
- `AJOUTER;id;nom;description` : ajoute un objet.
- `RECHERCHER;id` : recherche un objet par ID.
- `SUPPRIMER;id` : supprime un objet spécifique.
- `SUPPR_TOUT` : supprime tous les objets de la base.
- `AFFICHER_TOUS` : affiche tous les objets stockés.
- `QUITTER` : met fin à la session et déconnecte l'utilisateur.

Sauvegardes des données:

Les objets et les utilisateurs sont sérialisés dans des fichiers (`*.ser`) pour garantir la persistance entre les exécutions.

Nous pouvons découper la problématique de notre base de données en cinq sous-systèmes :

- La base de donnée orientée objet (classe BaseDeDonnees): gère le stockage, l'ajout,la recherche et la suppression des objets.
- La gestion des utilisateur(classe GestionUtilisateurs) : gère l'enregistrement et l'authentification des utilisateurs,avec un stockage d'information dans un fichier (*.serv).
- Le serveur multiclients (classe Serveur,ClientHandler) : permet d'accepter des connexions multiples, et de déléguer la gestion à un thread par client.

- Client interactif (classe Client) :permet de fournir une interface console pour envoyer des commandes au serveur
- Modèle de données(classes ObjetStocke,Utilisateur) :permet de représenter les entités manipulées (objet et utilisateur).

Description des classes principales :

Classe BaseDeDonnée:

Contient une HashMap<String, ObjetStocke>, la liste des objets stockés dans la base de données. Possèdent toutes les méthodes correspondant au commande CRUD que l'utilisateur peut entrer, soit ajouterObjet(), supprimerObjet(), rechercherObjet(), supprimerTousLesObjets(), getTousLesObjets().Ces dernières sont appelées dans la classe ClientHandler.

Classe GestionUtilisateurs:

Contient un HashMap<String, String> pour stocker le nom de l'utilisateur et son mot de passe. Possède toutes les méthodes gèrant les l'utilisateurs tel que ajouterUtilisateur(),verifierUtilisateur(),sauvegarderUtilisateurs() et chargerUtilisateurs(). Cette classe est utilisée par ClientHandler.

Classe Serveur:

Utilise la classe BaseDeDonnee et GestionUtilisateur,créeant ainsi deux instances.

Classe ClientHandler:

Utilise les classes BaseDeDonnee et GestionUtilisateur. Lors de la phase d'identification du client elle appelle les méthodes verifierUtilisateur() et ajouterUtilisateur(). Après l'authentification du client, elle fait appel au methodes de la classe BaseDeDonnee selon qu'elle commande à été rentré par l'utilisateur.

Classe Client:

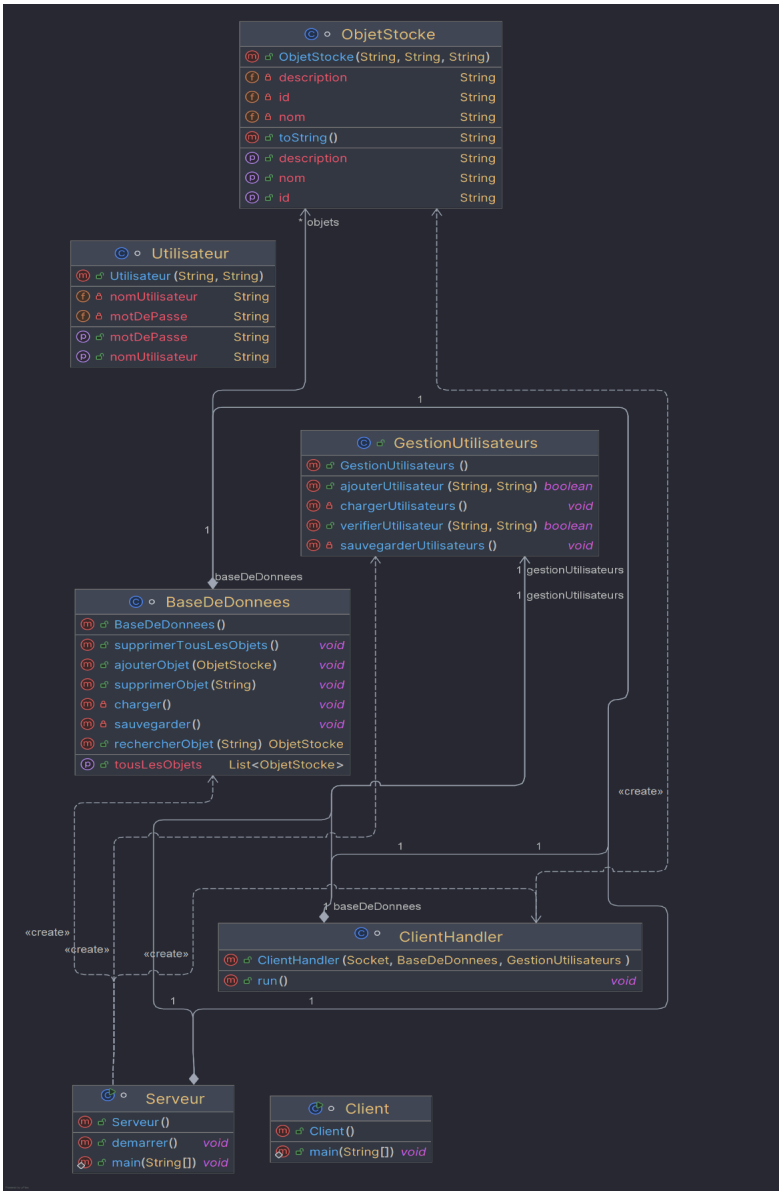
Application console dialoguant avec le serveur par sockets.Elle affiche les réponses et envoie les commandes entrées par l'utilisateur au serveur.

Classe ObjetStocke:

Classe représentant un objet à stocker avec le champ suivant : id, nom, description. Utilisé par la classe BaseDeDonnée dans un HasMap<> pour effectuer toutes ses méthodes.

II. Description des structures de données et Architecture logicielle

Diagramme Uml du projet Base de données orienté:



Les différentes couche de développement de notre projet est sous la forme:

1. Couche de présentation (interface utilisateur / client)

2. Couche de communication (réseau)

3. Couche métier (logique de traitement dans le serveur)

4. Couche d’accès aux données (fichiers ou base de données, sérialisation)

Voyons comment les couches sont organisée :

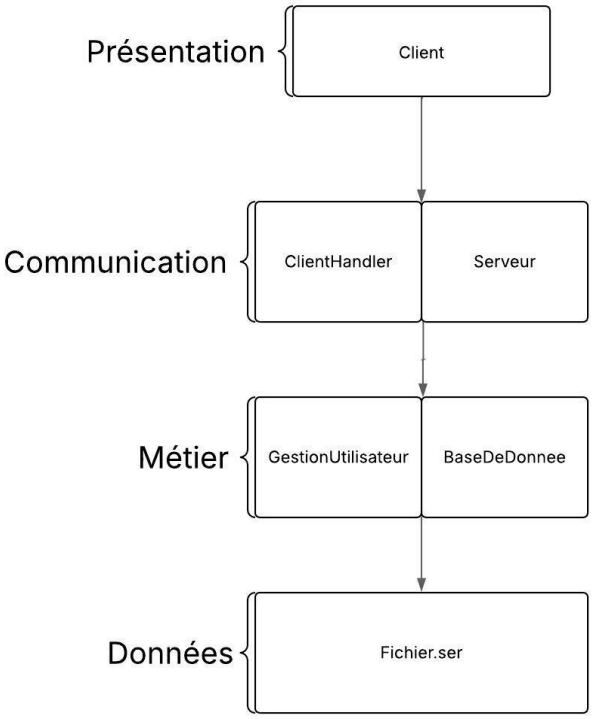
Donc, dans la couche 1, nous avons l’interface avec l’utilisateur qui se fait dans la ligne de commande, c’est-à-dire dans le cmd. Les classes qui sont concernées sont : Client et ClientHandler. Le client doit entrer les commandes qu’il veut effectuer en fonction de la commande, des réponses lui sont données.

Du côté des services effectués, nous avons donc, dans la communication, la classe serveur qui va effectuer la connexion des clients. Qui arrive sous forme de thread, elle permet l'accès au réseau des clients, donc à la base de données.
La forme de Thread des clients va nous permettre de faire de la gestion de concurrence sur les données de notre base, que ce soit les utilisateurs ou les objets. La classe clientHandler permet aussi la gestion des clients, donc de leurs interactions avec la base de données.

Dans la couche métier, donc le traitement des commandes, nous avons GestionUtilisateur et BaseDeDonnes qui sont nos dossiers de stockage de notre BD. Ces classes vont interagir sur les données avec leurs méthodes associées que nous verrons plus tard dans ce rapport.

Les classes utilisateur et ObjetStocke sont des classes sérialisées, ce sont donc ces données qui seront manipulées dans notre base de données.

Exemple de diagramme :



III.Spécification et description des classes principales

Nous allons voir dans cette partie les classes principales du projet avec leurs spécifications par rapport au code, comment nous avons choisi de les implémenter dans notre projet.

Les attributs de notre classe sont :

```
private Map<String, ObjetStocke> objets = new HashMap<>();
private final String fichierSauvegarde = "base_de_donnees.ser";
```

Les attributs de cette classe sont une Map de String qui revient à notre ID de l'objets associé, et un objets de type ObjetStocke.
On associe aussi le fichier de la base de données fichierSauvegarde.

Commençons d'abord par la classe BaseDeDonnes, cette classe permet la gestion des objets présents dans notre base de données, c'est-à-dire que l'on va directement

charger les données qui sont présentes dans notre dossier de base de données, pour cela on va créer un flux de type :

```
ObjectInputStream in = new ObjectInputStream(new
FileInputStream(fichierSauvegarde)
```

Donc la lecture du fichier se fait avec FileInputStream qui permet la lecture d'octets dans un fichier.
Ce fichier va être rendu persistant car on va associer le flux au fichier, grâce à la classe ObjectOutputStream et ObjectInputStream.
Une fois cela fait, nous récupérerons l'objet que nous ajoutons à notre liste d'objets :

```
objets = (Map<String, ObjetStocke>) in.readObject();
```

La liste d'objets se fait avec Map qui n'est pas synchronisée mais plus rapide dans la restitution de données, ce que nous avons préféré au HasHtable
L'appel de cette méthode se fait directement dans le constructeur de BaseDeDonnes.

Pour la sauvegarde de la base de données, nous procédons de la même façon, et la méthode qui nous permet de recharger la base de données.

```
out.writeObject(objets);
```

Cette méthode qui nous permet donc l'utilisation de :

```
public synchronized void ajouterObjet(ObjetStocke objet)
public synchronized void supprimerObjet(String id)
```

Ces méthodes permettant l'ajout et la suppression des objets se fait aussi avec les méthodes associées à la class Map :
Pour l'ajout des objets :

```
objets.put(objet.getId(), objet);
```

Ici l'ajout d'un objet est fait simplement avec son id et l'objet en lui-même.
Nous avons aussi fait le choix que si on ajoute un objet sur un objet déjà existant cela l'écrase. Nous verrons ce cas de figure ultérieurement.

Ensuite nous appelons la méthode Sauvegarder.

```
sauvegarder();
```

Ce qui nous garantit l'actualisation de notre base de données en temps réel et donc le maintien du serveur actif.

Continuons avec la classe ClientHandler, nous allons tout d'abord voir les attributs de cette classe :

```
private Socket socket;
private BaseDeDonnees baseDeDonnees;
```

```
private GestionUtilisateurs gestionUtilisateurs;
private boolean authentifie = false;
private String utilisateurActuel = null;
```

L'attribut baseDeDonnes est une instance de la classe vue précédemment.
L'attribut socket lui est : est une abstraction logicielle qui permet à un processus de communiquer avec un autre processus situé sur une machine différente.
Et comme dernier attribut notifiable, nous avons gestionUtilisateur qui va nous permettre de gérer notre liste d'utilisateurs présente dans notre base de données.

Nous avons donc un constructeur de base qui prend nos attributs en paramètre :

```
public ClientHandler(Socket socket, BaseDeDonnees baseDeDonnees,
GestionUtilisateurs gestionUtilisateurs) {
    this.socket = socket;
    this.baseDeDonnees = baseDeDonnees;
    this.gestionUtilisateurs = gestionUtilisateurs;
}
```

Ensuite l'ensemble de la classe ClienHandler se fait dans la méthode run.
Nous allons regarder cela plus en détail.

```
(ObjectInputStream in = new
ObjectInputStream(socket.getInputStream());
    ObjectOutputStream out = new
ObjectOutputStream(socket.getOutputStream())
```

On associe donc des flux depuis notre socket. In et out.
Le format des commandes se fait de cette manière :

```
out.writeObject("Bienvenue, " + utilisateurActuel + " ! Entrez
une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id
|SUPPR_TOUT | AFFICHER_TOUS | QUITTER ) :");
```

Dans un tableau de caractères divisé en différentes parties, on récupère ce que l'utilisateur va entrer.

```
String commande = (String) in.readObject();
String[] parts = commande.split(";");
```

Pour l'authentification nous procéder avec une boucle while avec condition :

```
out.writeObject("Veuillez vous connecter (LOGIN;nom;motdepasse)
ou créer un compte (REGISTER;nom;motdepasse)");
out.flush();
```

Il nous suffit donc de regarder quelle commande est rentrer :

```
if (parts[0].equalsIgnoreCase("LOGIN")) {
```

```
    if (gestionUtilisateurs.verifierUtilisateur(parts[1],
parts[2])) {
        authentifie = true;
        utilisateurActuel = parts[1];
        out.writeObject("Connexion réussie !");
    } else {
        out.writeObject("Échec de l'authentification.");
    }
}
```

On appelle une vérification des utilisateurs déjà présents ou pas dans la base de données, si tout est valide on peut passer à la deuxième étape qui sont les commandes entrées.
Une fois l'authentification réussie.
En fonction de la commande entrer, nous effectuons les ajouts, affichage, suppressions associés à la commande. Par exemple, pour l'ajout, nous effectuons :
nous avons donc un switch avec lequel on vérifie la première partie de la commande :

```
switch (parts[0].toUpperCase())
```

et en fonction de la commande on effectue différentes opération :

```
case "AJOUTER" :
case "RECHERCHER":
case "SUPPRIMER":
case "SUPPR_TOUT":
case "AFFICHER_TOUS":
case "QUITTER":
```

la commande pour ***ajouter*** un objet est de cette forme :

```
AJOUTER;id;nom;desc
```

On va donc récupérer chaque partie de notre commande renter et l'ajouter a la liste des objets avec la création d'un nouvel objet.
Le constructeur d' objets ce fait de cette manière :

```
public ObjetStocke(String id, String nom, String description)
```

Nous avons donc notre objet :

```
ObjetStocke objet = new ObjetStocke(parts[1], parts[2],
parts[3]);
```

il suffit ici de l'ajouter à notre base de donnée :

```
baseDeDonnees.ajouterObjet(objet);
```

la commande ***rechercher*** permet la recherche d'un objet dans la base de donnée
il suffit de récupérer le numéro de l'id de l'élément rechercher :

```
ObjetStocke objet = baseDeDonnees.rechercherObjet(parts[1]);
```

```
out.writeObject((objet == null) ? "Objet non trouvé" :
objet.toString());
```

Passons maintenant à la classe GestionUtilisateurs.
Nous avons comme attributs:

```
private Map<String, String> utilisateurs = new HashMap<>();
private final String fichierUtilisateurs = "utilisateurs.ser";
```

Donc une liste d'utilisateurs sous forme de Map et le fichier de stockage des utilisateurs de notre base de données.
Le constructeur lui est sans paramètre il va directement charger l'ensemble des utilisateurs avec la méthode chargerUtilisateur().

```
public GestionUtilisateurs() {
   chargerUtilisateurs();
}
```

Regardons donc plus en détails les méthodes associées à cette classe :
La méthode sauvegarderUtilisateurs fonctionne sur le même principe que sauvegarder dans la Classe BaseDeDonnes.

```
(ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream(fichierUtilisateurs))
```

```
out.writeObject(utilisateurs)
```

Le chargement de la liste des utilisateurs dans notre attribut Map(utilisateurs) aussi (chargerUtilisateurs):

```
try (ObjectInputStream in = new ObjectInputStream(new
FileInputStream(fichierUtilisateurs))) {
   utilisateurs = (Map<String, String>) in.readObject();
```

Les deux dernières méthodes sont ajouterUtilisateur et verifierUtilisateur en boolean la première fait l'ajout et la vérification d'un nom d'utilisateur si il est déjà utilisée ou pas :

```
if (utilisateurs.containsKey(nom)) {
   return false; // L'utilisateur existe déjà
}
utilisateurs.put(nom, motDePasse);
sauvegarderUtilisateurs();
return true;
```

et pour verifierUtilisateur:

```
return utilisateurs.containsKey(nom) &&
utilisateurs.get(nom).equals(motDePasse);
```

On vérifie que le nom est présent dans notre liste d'utilisateurs et aussi que le mot de passe associé au nom existe.

Nous allons finir la présentation détaillée des méthodes et des classes importantes par la classe client.

La classe client crée un socket avec le port et l'adresse IP sélectionnée.
On crée des flux out et in.

```
Socket socket = new Socket(HOST, PORT);
    ObjectOutputStream out = new
ObjectOutputStream(socket.getOutputStream());
    ObjectInputStream in = new
ObjectInputStream(socket.getInputStream());
```

ObjectOutputStream,ObjectInputStream: permettent d'envoyer et recevoir des objets Java (ici des String, des List, etc.)

La première boucle dans le code permet de faire l'authentification du client qui se connecte à la base de données.

```
String message = (String) in.readObject();
System.out.println(message); // Message de connexion

String commande = scanner.nextLine();
out.writeObject(commande);
out.flush();

String reponse = (String) in.readObject();
System.out.println("Réponse du serveur: " + reponse);

if (reponse.contains("Connexion réussie")) {
   break; // Passe à la boucle de commandes
```

Dans la deuxième boucle on demande à l'utilisateur de rentrer sa commande donc :

```
String message = (String) in.readObject();
System.out.println(message); // Invite de commande

String commande = scanner.nextLine();
out.writeObject(commande);
out.flush();
```

```
Object  reponse = in.readObject();
```

on doit vérifier que l'objet qui est demandé est un string ou une liste d'objets

```
if (reponse instanceof List) {//verif si c'est une liste
   List<String> objets = (List<String>) reponse;
```

et si notre liste n'est pas vide on l'affiche avec :

```
System.out.println("Objets dans la base :");
for (Object obj : objets) {
   System.out.println(obj);
}
```

Pour résumer nous avons vu dans la globalité les principales classes et fonctionnalités qu'elles apportent au projet et la façon dont nous avons implémenté notre code a notre façon, comment nous avons interpréter le projet et donc comment nous avons fait dans la gestion de nos classes.

IV.Test

L'utilisateur une fois connecté au serveur doit s'identifier avec la commande LOGIN;nom;mot de passe ou s'enregistrer avec REGISTER;nom;mot de passe.

```
Veuillez vous connecter (LOGIN;nom;motdepasse) ou créer un compte (REGISTER;nom;motdepasse)
REGISTER;martin;1234
Réponse du serveur: Utilisateur créé avec succès.
Veuillez vous connecter (LOGIN;nom;motdepasse) ou créer un compte (REGISTER;nom;motdepasse)
LOGIN;martin;1234
Réponse du serveur: Connexion réussie !
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
```

Après l'enregistrement il a accès au commande lui permettant d'effectuer divers requêtes sur la base de données.Comme la base de donnée est actuellement vide on va y ajouter deux objets que l'on affichera avec la commande AFFICHER_TOUT.

```
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AJOUTER;0;chat;petit animal de compagnie
Réponse du serveur: Objet ajouté avec succès.
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AJOUTER;1;chien;animal de compagnie
Réponse du serveur: Objet ajouté avec succès.
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AFFICHER_TOUT
Objets dans la base :
ObjetStocke{id='0', nom='chat', description='petit animal de compagnie'}
ObjetStocke{id='1', nom='chien', description='animal de compagnie'}
```

On peut supprimer un objet de notre base de donnée en renseignant son id, à noter que si on créer un objet ayant un id déjà renseigné dans la base de donnée l'ancien objet sera remplacé par le nouveau.

```
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AJOUTER;0;grenouille;petit amphibien
Réponse du serveur: Objet ajouté avec succès.
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AFFICHER_TOUT
Objets dans la base :
ObjetStocke{id='0', nom='grenouille', description='petit amphibien'}
ObjetStocke{id='1', nom='chien', description='animal de compagnie'}
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
SUPPRIMER;0
Réponse du serveur: Objet supprimé.
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AFFICHER_TOUT
Objets dans la base :
ObjetStocke{id='1', nom='chien', description='animal de compagnie'}
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
```

On peut aussi effectuer une recherche par id

```
RECHERCHER;1
Réponse du serveur: ObjetStocke{id='1', nom='chien', description='animal de compagnie'}
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
```

On peut supprimer tout les objet de notre base de donnée avec la commande SUPPR_TOUT

```
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AJOUTER;0;renard;petit animal
Réponse du serveur: Objet ajouté avec succès.
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AJOUTER;2;corbeau;petit volatile
Réponse du serveur: Objet ajouté avec succès.
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AFFICHER_TOUT
Objets dans la base :
ObjetStocke{id='0', nom='renard', description='petit animal'}
ObjetStocke{id='1', nom='chien', description='animal de compagnie'}
ObjetStocke{id='2', nom='corbeau', description='petit volatile'}
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
SUPPR_TOUT
Réponse du serveur: Tous les objets ont été supprimés.
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AFFICHER_TOUT
Réponse du serveur: Aucun objet enregistré.
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
```

Si un deuxième utilisateur apporte des modifications sur la base de données elle sera directement visible par le premier utilisateur.

Client 1:

```
Réponse du serveur: Connexion réussie !
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AFFICHER_TOUT
Réponse du serveur: Aucun objet enregistré.
```

Client 2:

```
Veuillez vous connecter (LOGIN;nom;motdepasse) ou créer un compte (REGISTER;nom;motdepasse)
REGISTER;didier;azerty
Réponse du serveur: Utilisateur créé avec succès.
Veuillez vous connecter (LOGIN;nom;motdepasse) ou créer un compte (REGISTER;nom;motdepasse)
LOGIN;didier;azerty
Réponse du serveur: Connexion réussie !
Bienvenue, didier ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AJOUTER;0;chaise;meuble
Réponse du serveur: Objet ajouté avec succès.
```

Client 1 :

```
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
AFFICHER_TOUT
Objets dans la base :
ObjetStocke{id='0', nom='chaise', description='meuble'}
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
```

Une fois terminé on peut quitter la base de données

```
Bienvenue, martin ! Entrez une commande (AJOUTER;id;nom;desc | RECHERCHER;id | SUPPRIMER;id |SUPPR_TOUT | AFFICHER_TOUT | QUITTER ) :
QUITTER
Réponse du serveur: Déconnexion réussie.

Process finished with exit code 0
```

V.Conclusion

Dans ce projet de base de données orientée objet a permis de mettre en œuvre une solution complète de gestion d'objets persistants en environnement distribué. À travers la conception et le développement d'un serveur multi-utilisateurs et d'un client terminal interconnecté, nous avons pu explorer les différentes facettes de la communication réseau, de la sérialisation Java, de la gestion de la concurrence et de l'organisation logicielle en couches fonctionnelles.

Le serveur assure efficacement la gestion des connexions simultanées, l'authentification des utilisateurs à la base de données et la persistance des données, tandis que le client permet une interaction fluide avec les collections d'objets distantes, en respectant les principes du modèle CRUD.

Ce travail nous a offert une expérience concrète de développement logiciel collaboratif, en intégrant à la fois des aspects théoriques vus en cours et des défis techniques rencontrés lors de la mise en œuvre.